

Step 4 — Object Relationships

This document explains the object relationships used in the **Cinema Ticket Booking System**.

To choose the appropriate relationship, we use the **Lifetime Test**:

"If the main object is destroyed, does the smaller object also die with it, or does it survive independently?"

1. Simple Guide to Object Relationships

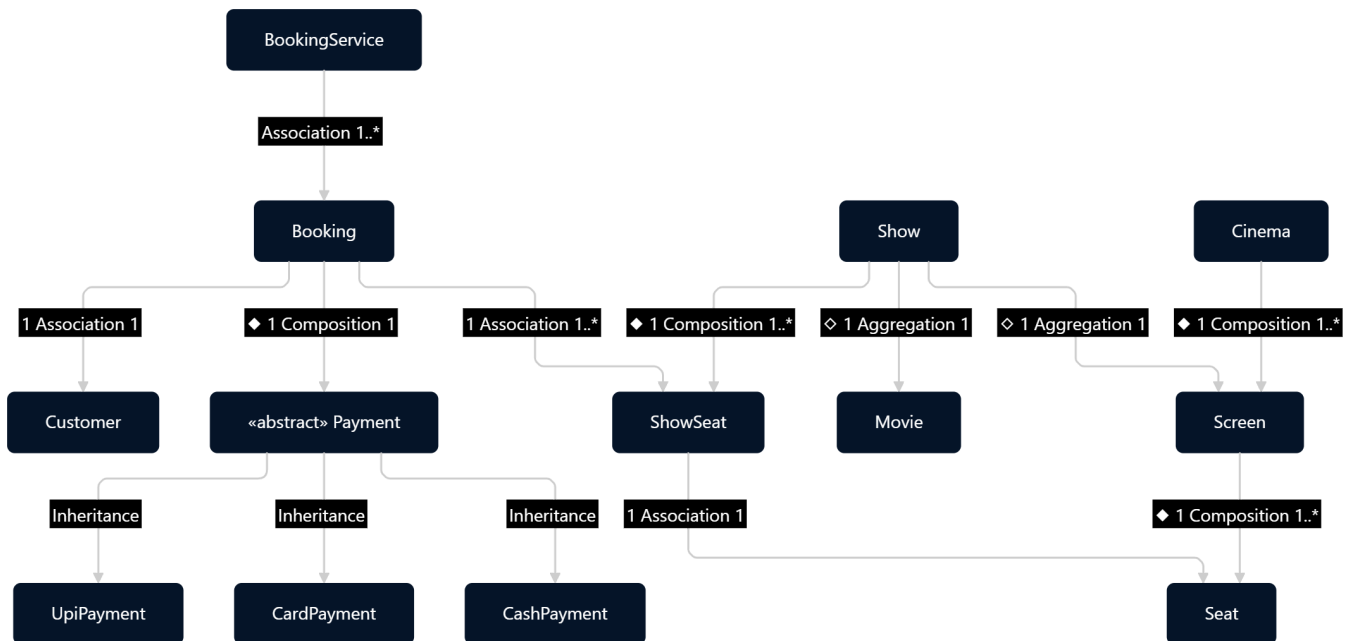
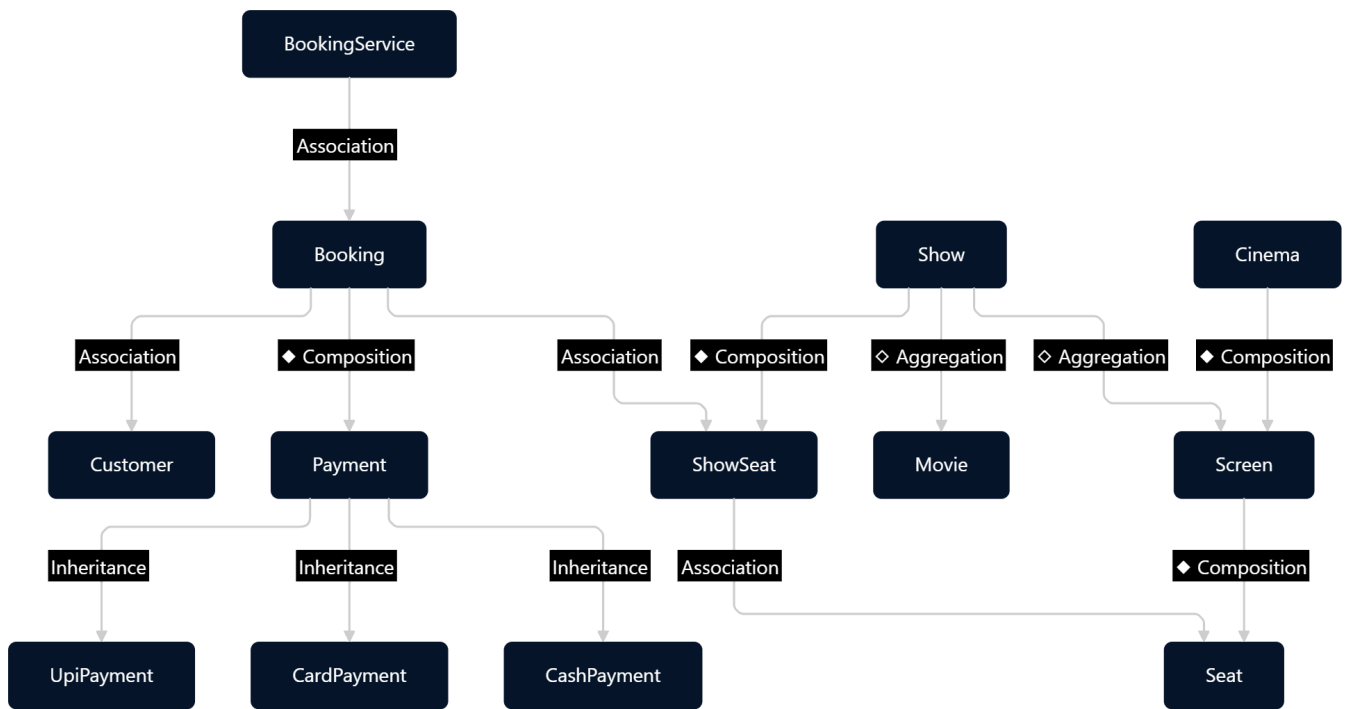
Relationship Type	Meaning	Strength	Lifetime Test	Real-World Example
Composition	Part-Of	Strong	If the parent is destroyed, the child is also destroyed.	A House and a Room
Aggregation	Has-A	Weak	If the parent is destroyed, the child can survive independently.	A Library and a Book
Association	Uses/Interacts-With	Independent	Both objects can exist independently and simply interact with each other.	A Customer and an ATM
Inheritance	Is-A	Parent/Child	One class is a specialized version of another class.	An Animal and a Dog

2. System Relationships and Justification

Pair	Relationship	Justification
Cinema — Screen	Composition	A <code>Screen</code> is a permanent part of a <code>Cinema</code> . If the cinema is removed from the system, its screens are also removed. Therefore, the <code>Screen</code> has a strong lifecycle dependency on the <code>Cinema</code> .

Pair	Relationship	Justification
Screen — Seat	Composition	A <code>Seat</code> belongs to a particular <code>Screen</code> . If the screen is removed from the system, its seats are also removed. Therefore, <code>Seat</code> is composed within <code>Screen</code> .
Show — Movie	Aggregation	A <code>Show</code> uses a <code>Movie</code> , but the movie can exist independently. If a 6:00 PM show is cancelled, the movie still exists and can be scheduled for another show.
Show — Screen	Aggregation	A <code>Show</code> is scheduled on a <code>Screen</code> , but the screen exists independently. If a show is cancelled, the screen can still be used for other shows.
Show — ShowSeat	Composition	A <code>ShowSeat</code> represents the seat availability/status for one specific <code>Show</code> . If the show is deleted, its <code>ShowSeat</code> records no longer have a purpose and are also deleted.
Booking — Customer	Association	A <code>Customer</code> makes a <code>Booking</code> , but both objects can exist independently. If a booking is cancelled or deleted, the customer still exists.
Booking — ShowSeat	Association	A <code>Booking</code> selects or reserves <code>ShowSeat</code> objects. If the booking is cancelled, the show seats continue to exist and can become available again.
Booking — Payment	Composition	A <code>Payment</code> transaction is created for a specific <code>Booking</code> . In this simplified system model, if the booking is permanently deleted, its associated payment record is also removed.
Payment — UpiPayment	Inheritance	<code>UpiPayment</code> is a specialized type of <code>Payment</code> . Similarly, <code>CardPayment</code> and <code>CashPayment</code> can inherit from the <code>Payment</code> class.
BookingService — Booking	Association	<code>BookingService</code> manages the creation and processing of bookings, but bookings can exist independently. If the service is stopped or removed, existing bookings remain stored in the database.

3. FINAL Relationships Diagram



Note: In a real database-backed system, `Booking - Payment` may also be modeled as an **Association** rather than Composition if payment records must be retained for auditing, refunds, or financial history even after a booking is deleted.